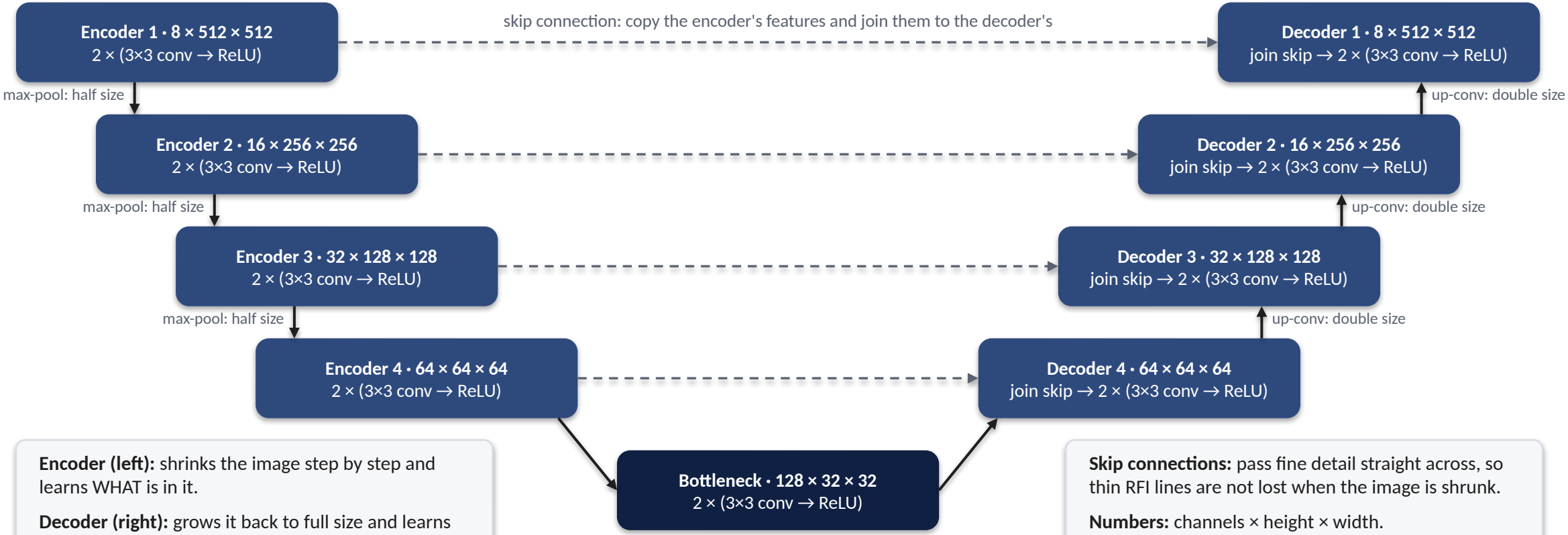
➢Starting point: a basic U-Net

▼ Input: spectrogram, 1 × 512 × 512 (time × frequency)

Output: an RFI probability for every pixel ▲



Encoder (left): shrinks the image step by step and learns WHAT is in it.

Decoder (right): grows it back to full size and learns WHERE it is.

Skip connections: pass fine detail straight across, so thin RFI lines are not lost when the image is shrunk.

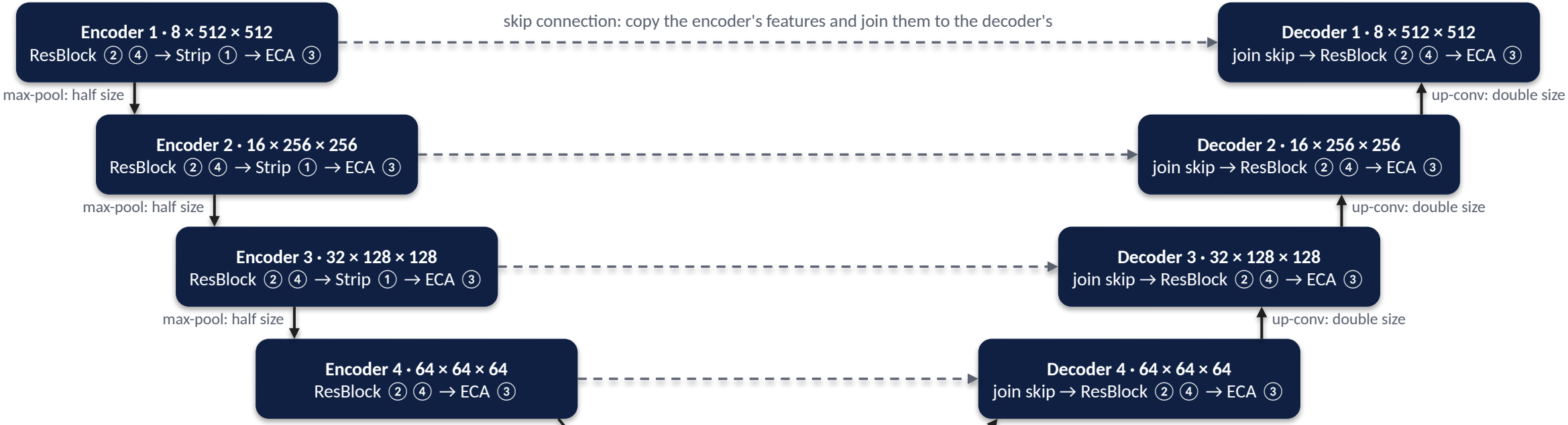
Numbers: channels × height × width.

Drawn at our model's sizes so slides 1 and 2 line up box for box. Akeret's tf_unet (our baseline) is this design, but shrinks only twice (32 → 64 → 128 channels, 465,986 parameters) and trims the border, so 512 × 512 in gives 472 × 472 out.

➢Our model: the same U, changed inside

▼ Input: spectrogram, 1 × 512 × 512 — no border trimming ⑦

1×1 conv → 2 raw scores → softmax ⑤ → RFI probability ▲



① **Strip convolutions** — 7/11/21-px windows (enc. 1–3, bottleneck)

② **Residual shortcut** — inside every ResBlock

③ **ECA attention** — after every encoder and decoder block

④ **GroupNorm** — in every ResBlock and strip (tf_unet: none)

⑤ **Raw scores** — no ReLU before softmax (tf_unet has one)

⑥ **Dice + cross-entropy loss** (tf_unet: cross-entropy + L2 penalty)

⑦ **Same padding** — every pixel gets a prediction

⑧ **No class weighting** — tf_unet has none either; our earlier version used it and scored lower

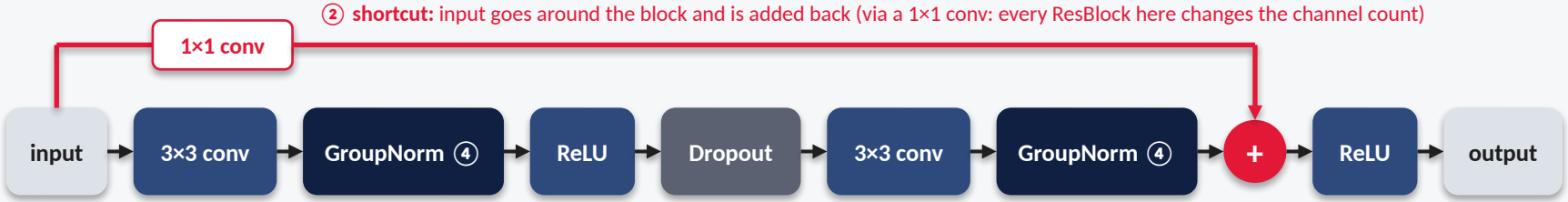
⑨ **Size**: shrinks 4 times, 8 → 128 channels, 593,842 parameters

Red = the three added modules ① – ③

➢Inside one level, step by step

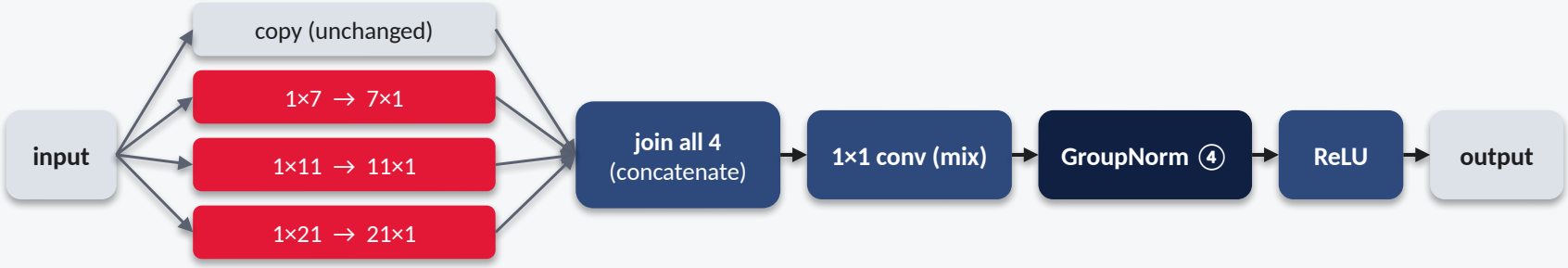
ResBlock ② ④

Two 3×3 convolutions, each tidied by GroupNorm; the input is added back at the end. Dropout works in training only.



Strip module ①

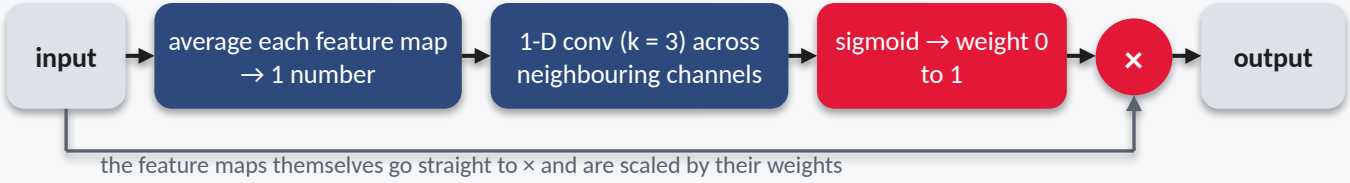
Looks at wider areas (7, 11, 21 px) cheaply: a thin filter along frequency, then one along time.



Each path = 1×K along frequency, then K×1 along time, each channel on its own → together it covers a K×K square (measured)

ECA attention ③

Gives each feature map a weight between 0 and 1, then scales it.



the feature maps themselves go straight to × and are scaled by their weights

From output to RFI mask, and how it learns

Using the model
after training

spectrogram → scale to 0–1 (fixed range) → Our model → probability 0–1 for each pixel → threshold (picked on validation images) → RFI mask: yes / no per pixel

Training
28,000 steps, one image per step

model's probabilities
AOFlagger's RFI mask
→ **loss = cross-entropy + Dice** ⑥ how wrong the model is; no class weighting ⑧
→ **Adam optimiser** nudges all 593,842 numbers to lower the loss
→ **Every 1,400 steps** check on 150 validation images; keep the best version

How it is scored: 109 test images labelled by a human expert; F1 at the best threshold (the standard for this benchmark); 3 runs each, mean ± spread.

Our model 0.6603 ± 0.0040 vs Akeret's tf_unet 0.5482 ± 0.0139 → about +0.11.

Which parts matter? Removing ① ② ③ gives 0.6585 — not significant (p = 0.64). Removing ④ as well gives 0.6495 — also not significant (p = 0.48).

So the gain over tf_unet sits mainly in ⑤ ⑥ ⑦ ⑨ (raw output, Dice loss, same padding, size and training setup) — not yet tested one at a time.